PyTorch ile Derin Öğrenme Algoritmaları

PyTorch, dinamik hesaplama grafikleri (Dynamic Computational Graphs) kullanarak esnek, hızlı ve modüler yapay zekâ modelleri geliştirilmesine olanak tanıyan, Torch kütüphanesi tabanlı açık kaynak makine öğrenmesi kütüphanesidir. **Meta AI Araştırma Laboratuvarı (FAIR)** tarafından geliştirilen bu platform hem akademik araştırmalarda hem de endüstriyel devlerde dünya standardı olarak kabul görmektedir.

Temel Özellikler ve Yetenekler:

PyTorch'u diğer kütüphanelerden ayıran temel teknik avantajlar şunlardır:

- **Dinamik Hesaplama Grafiği (Autograd):** Kod çalışma esnasında (*on-the-fly*) grafik oluşturur. Bu durum, özellikle değişken uzunluklu girdilerle çalışan Doğal Dil İşleme (NLP) modellerinde ve karmaşık ağ yapılarında muazzam bir esneklik sağlar.
- **Donanım Hızlandırma (GPU Desteği):** Tensor hesaplamalarını NVIDIA CUDA ve diğer hızlandırıcılar üzerinde yüksek performansla koşturarak eğitim sürelerini optimize eder.
- **Eager ve Grafik Modları (TorchScript):** Geliştirme aşamasında Pythonik bir yapı sunan "Eager Mode" ile kolay hata ayıklama (debugging) imkânı verirken; TorchScript ile modelleri üretim (production) ortamı için yüksek performanslı grafik moduna dönüştürür.
- **Dağıtılmış Eğitim (Distributed Training):** “torch.distributed” modülü sayesinde, modellerin birden fazla GPU veya sunucu üzerinde ölçeklenebilir şekilde eğitilmesini sağlar.

Uygulama ve Kullanım Alanları:

- **Bilgisayarlı Görme:** Nesne tespiti, yüz tanıma ve görüntü segmentasyonu.
- **Doğal Dil İşleme (NLP):** Çeviri sistemleri, metin sınıflandırma ve büyük dil modelleri.
- **Ses İşleme**: Konuşmadan metne (STT) ve metinden konuşmaya (TTS) sentezleme.
- **Zaman Serisi Analizi**: Borsa tahmini, anomali tespiti ve talep öngörüleri.

Endüstriyel Referanslar ve Gerçek Dünya Uygulamaları

- **Tesla:** Otonom sürüş sistemlerindeki görüntü işleme ve karar verme algoritmalarında.
- **Meta:** Sosyal medya içerik öneri sistemleri ve çeviri araçlarında.
- **Uber:** Otonom araçlar için nesne tespiti ve rota optimizasyonunda.
- **NVIDIA:** Derin öğrenme donanımlarının optimizasyonu ve yeni mimarilerin geliştirilmesinde.

Ekosistem ve Bulut Entegrasyonu: PyTorch, sadece bir kütüphane değil, geniş bir araç setidir:

- **Sağlam Ekosistem:** torchvision (görüntü), torchaudio (ses) ve torchtext (metin) ile projeye özel hazır araçlar sunar.
- **Bulut Desteği:** AWS, AWS, Google Cloud ve Microsoft Azure ile tam entegre çalışarak modellerin kolayca ölçeklenmesini ve dağıtılmasını (deployment) sağlar.
- **Hugging Face Desteği:** Modern NLP ve Transformer modellerine en hızlı erişim PyTorch ekosistemi üzerinden gerçekleşir.

PyTorch vs Diğer Derin Öğrenme Kütüphaneleri:

Özellik	Keras	TensorFlow	PyTorch
Kullanım Kolaylığı	Çok kullanıcı dostu, yüksek seviyeli API.	Daha düşük seviye, esneklik odaklı	Pythonic, doğal bir API yapısı.
Performans	TensorFlow tabanlı olduğu için yüksek performans.	Yüksek performans ve geniş model desteği.	Yüksek performanslı, GPU desteği.
Esneklik	Orta derecede esneklik, basit modeller için ideal.	Çok esnek, düşük seviyeli kontrol.	Çok esnek, düşük seviyeli kontrol.
Model Eğitim Hızı	Hızlı model geliştirme ve eğitim.	Büyük veri ve büyük modellerde optimize edilmiş.	Eğitim hızı optimize edilebilir.
Desteklenen Cihazlar	CPU ve GPU.	CPU, GPU, TPU.	CPU, GPU, TPU.
Topluluk ve Dokümantasyon	Geniş topluluk, zengin dokümantasyon.	En büyük topluluk ve kapsamlı dokümantasyon.	Aktif topluluk ve detaylı dokümantasyon.
Dağıtık Eğitim	Temel destek.	Güçlü destek, veri paralel ve model paralel.	Temel destek.
Öğrenme Eğrisi	Düşük, yeni başlayanlar için ideal.	Daha dik, düşük seviyeli operasyonlar.	Orta düzey, Python bilginiz varsa kolay.
İleri Seviye Özelleştirme	Kısıtlı, yüksek seviyede API.	Çok esnek, ileri düzey kontrol ve özelleştirme.	Yüksek esneklik, araştırma ve prototipler için ideal.
Model Zenginliği	Sınırlı, hazır modellerle hızlı sonuçlar.	Çok geniş model yelpazesi.	Geniş, araştırma modelleri için ideal.
Popüler Kullanım Alanları	Hızlı prototipleme, küçük-orta ölçekli projeler.	Endüstriyel büyük ölçekli projeler.	Araştırma projeleri, dinamik veri işleme.

Temel Veri Yapısı: Tensörler (Tensors):

PyTorch ekosisteminde her türlü veri, Tensör adı verilen matematiksel objelerle temsil edilir. Tensörler, yapı olarak NumPy'daki ndarray yapısına benzer ancak iki kritik "süper güce" sahiptir:

Tensörlerin Ayırt Edici Özellikleri

1. GPU Entegrasyonu: Tensörler, doğrudan NVIDIA GPU'lar üzerinde tanımlanabilir ve işlenebilir. Bu sayede CUDA altyapısı kullanılarak devasa veri setleri üzerinde yüksek hızlı hesaplama yapılır.
2. Otomatik Türev (Autograd): Tensörler, üzerlerinde gerçekleştirilen işlemleri bir hesaplama grafiğinde tutar. Bu sayede derin öğrenmenin temeli olan gradyan (türev) hesaplamaları otomatik olarak gerçekleştirilir.

Uygulama Süreçlerinde Tensör Kullanımı:

- **Girdi Verileri:** Görseller, ses dalgaları veya metinler önce sayısal değerlere, ardından tensör formatına dönüştürülerek modele iletilir.
- **Model Parametreleri (Ağırlıklar):** Modelin eğitim sırasında öğrendiği tüm ağırlıklar (weights) ve sapmalar (biases) tensörlerde saklanır.
- **Tahmin Çıktıları:** Modelin ürettiği sınıflandırma veya regresyon sonuçları yine birer tensör objesidir.

PyTorch Python API ve Modüler Yapısı

PyTorch, modelleri inşa etmek ve eğitmek için hiyerarşik bir API yapısı sunar. Bu modüller, bir yapay zekâ modelinin mimarisinden hata hesaplamasına kadar her aşamayı kapsar.

Temel Bileşenler:

- **torch:** Tensör işlemleri ve temel matematiksel fonksiyonları barındıran ana modüldür.
- **torch.nn:** Sinir ağı katmanlarını (layers) ve yapısal modülleri içeren temel kütüphanedir.
- **torch.nn.functional:** Aktivasyon fonksiyonları ve kayıp hesaplamaları gibi işlemleri fonksiyonel düzeyde sunar.
- **torch.autograd:** Geriye yayılım (Backpropagation) sürecini yöneten otomatik türev mekanizmasıdır.
- **torch.optim:** Model parametrelerini güncellemek için kullanılan optimizasyon algoritmalarını (SGD, Adam vb.) içerir.

torch.nn Katman Tipleri ve Kullanım Amaçları

- **Linear Layers:** Tam bağlantılı (Fully-connected) yapılar; ANN mimarisinin temelidir.
- **Convolutional Layers:** Görüntü işleme ve özellik çıkarma süreçlerinde kullanılır.
- **Recurrent Layers:** RNN ve LSTM gibi zaman serisi ve NLP verileri için geri beslemeli yapılar.
- **Transformer Layers:** Multi-head attention mekanizmasıyla modern dil modellerinin (LLM) temelini oluşturur.
- **Non-linear Activations:** Sinir ağlarında doğrusal olmayan dönüşümleri uygulayan aktivasyon fonksiyonları. (ReLU ve Sigmoid gibi)
- **Normalization Layers:** Modelin daha hızlı ve stabil öğrenmesini sağlamak için Batch Normalization gibi teknikleri içerir.
- **Dropout Layers:** Bazı bağlantıları rastgele sıfırlayarak aşırı öğrenmeyi (Overfitting) engeller.
- **Pooling & Padding:** Özellik haritalarını küçültme (Pooling) ve boyut koruma (Padding) işlemleri.
- **Loss Functions:** Modelin hatasını ölçmek için kullanılan kayıp fonksiyonları- MSE (Mean Square Error) veya Cross Entropy gibi fonksiyonlar.

PyTorch Genişletilmiş Kütüphane Ekosistemi

- **Torchvision:** Bilgisayarlı görme (Computer Vision) için veri setleri ve önceden eğitilmiş modeller.
- **TorchAudio:** Ses sinyali işleme ve modelleme araçları.
- **TorchText:** Doğal Dil İşleme (NLP) için veri hazırlama ve modelleme.
- **TorchServe:** Modellerin üretim ortamına (Production) hızlıca dağıtılmasını sağlar.
- **TorchRec:** Büyük ölçekli öneri sistemleri (Recommendation Systems) için geliştirilmiştir.
- **PyTorch / XLA:** PyTorch modellerinin Google TPU donanımları üzerinde çalışmasını sağlar.

PROJE 1: MNIST El Yazısı Rakam Sınıflandırma

PyTorch ile Yapay Sinir Ağı (ANN) Uygulaması:

PyTorch kütüphanesi kullanılarak MNIST veri seti üzerinde el yazısı rakamların sınıflandırılmasını amaçlamaktadır. MNIST veri seti, 0–9 arasındaki rakamlardan oluşan 28x28 piksel boyutunda gri tonlamalı görüntüler içerir ve derin öğrenme alanında temel bir benchmark veri seti olarak kabul edilir. Bu çalışmada, çok katmanlı tam bağlantılı bir Artificial Neural Network –ANN mimarisi tasarlanmış ve modelin eğitim, test ve değerlendirme süreçleri gerçekleştirilmiştir.

1. Veri Hazırlama Süreci: Projenin ilk aşamasında MNIST veri seti torchvision.datasets modülü aracılığıyla indirilmiştir. Veriler üzerinde şu işlemler uygulanmıştır:

- Görüntüler tensöre dönüştürülmüştür (ToTensor)
- Normalize edilerek değerler -1 ile 1 arasına ölçeklenmiştir
- Eğitim ve test veri setleri ayrı ayrı yüklenmiştir
- DataLoader ile batch (64 örnek) yapısına dönüştürülmüştür

2. Model Mimarisi: Model, üç katmanlı tam bağlantılı bir yapay sinir ağıdır:

- ***Giriş Katmanı:*** 28x28 = 784 nöron
- ***1. Gizli Katman:*** 128 nöron + ReLU aktivasyonu
- ***2. Gizli Katman:*** 64 nöron + ReLU aktivasyonu
- ***Çıkış Katmanı:*** 10 nöron (0–9 rakam sınıfları)

Model mimarisi **"nn.Module"** sınıfından türetilmiştir ve forward propagation işlemi manuel olarak tanımlanmıştır. Aktivasyon fonksiyonu olarak **ReLU** (Rectified Linear Unit) kullanılmıştır. Çıkış katmanında softmax uygulanmamıştır çünkü CrossEntropyLoss fonksiyonu bunu dahili olarak gerçekleştirir.

3. Eğitim Süreci: Modelin eğitimi sırasında:

- ***Kayıp Fonksiyonu:*** CrossEntropyLoss
- ***Optimizasyon Algoritması:*** Adam
- ***Learning Rate:*** 0.001
- ***Epoch Sayısı:*** 10

Her epoch sonunda ortalama loss değeri hesaplanarak yazdırılmıştır. Ayrıca eğitim sürecinin görselleştirilmesi için loss grafiği oluşturulmuştur.

Eğitim adımları: Forward propagation → Loss hesaplama → Backpropagation → Ağırlık güncelleme

4. Test ve Değerlendirme: Model eval() moduna alınarak test veri seti üzerinde değerlendirilmiştir. Başarı ölçütü olarak **Accuracy (Doğruluk Oranı)** kullanılmıştır:

Accuracy = Doğru Tahmin Sayısı / Toplam Örnek Sayısı

Elde edilen sonuçlar modelin MNIST veri seti üzerinde genellikle %95'in üzerinde doğruluk sağladığını göstermektedir.

5. Donanım Kullanımı ve Projenin Önemi

Kod, CUDA destekli GPU mevcutsa otomatik olarak GPU kullanacak şekilde yapılandırılmıştır. Bu durum eğitim sürecini önemli ölçüde hızlandırmaktadır. Bu proje, derin öğrenmenin temel prensiplerini, PyTorch mimarisini, tensör yapısını ve otomatik türev (autograd) sistemini uygulamalı olarak göstermektedir. Ayrıca ANN mimarisinin sınıflandırma problemlerindeki performansını ortaya koyarak CNN ve daha gelişmiş derin öğrenme modelleri için temel oluşturmaktadır.

PROJE 2: CIFAR-10 Nesne Tanıma

PyTorch ile Evrişimli Sinir Ağı (CNN) Uygulaması:

PyTorch kütüphanesi kullanılarak CIFAR-10 veri seti üzerinde 10 farklı nesne kategorisinin (uçak, otomobil, kuş, kedi, geyik, köpek, kurbağa, at, gemi, kamyon) sınıflandırılmasını amaçlamaktadır. CIFAR-10 veri seti 32x32 piksel boyutunda RGB görüntüler içerir. Bu çalışma kapsamında, görüntü verileri üzerinde özellik çıkarımı yapabilen bir Convolutional Neural Network – CNN mimarisi tasarlanmış, modelin eğitim ve test süreçleri gerçekleştirilmiş ve performans analizi yapılmıştır.

1. Veri Hazırlama Süreci: Projenin ilk aşamasında CIFAR-10 veri seti torchvision.datasets modülü aracılığıyla indirilmiştir. Veri ön işleme sürecinde:

- Görüntüler tensör formatına dönüştürülmüştür. (ToTensor)
- RGB kanalları normalize edilmiştir. (mean=0.5, std=0.5)
- Eğitim ve test veri setleri ayrılmıştır.
- DataLoader ile batch (64 örnek) yapısına dönüştürülmüştür.

2. Model Mimarisi (CNN Yapısı): Model, iki evrişim bloğu ve iki tam bağlantılı katmandan oluşmaktadır:

- **Evrişim Katmanları (Conv1 & Conv2):** Görüntülerden kenar, şekil ve nesne parçaları gibi anlamlı uzamsal özellikleri çıkarır.
- **Aktivasyon & Havuzlama:** Her evrişim sonrası **ReLU** aktivasyonu ve boyutu küçülterek önemli özellikleri koruyan **MaxPool** uygulanmıştır.
- **Regülarizasyon:** Aşırı öğrenmeyi (overfitting) engellemek için %20 oranında **Dropout** katmanı eklenmiştir.
- **Sınıflandırma:** Son aşamada 4096'dan 10 sınıfa indirgeme yapan tam bağlantılı katmanlar kullanılmıştır.

Evrişim katmanları görüntülerden kenar, şekil ve nesne parçaları gibi anlamlı özellikler çıkarırken, tam bağlantılı katmanlar sınıflandırma işlemini gerçekleştirir. Çıkış katmanında softmax manuel uygulanmamıştır çünkü CrossEntropyLoss bunu dahili olarak hesaplamaktadır.

3. Eğitim Süreci: Model aşağıdaki hiperparametrelerle eğitilmiştir:

- ***Kayıp Fonksiyonu:*** CrossEntropyLoss
- ***Optimizasyon Algoritması:*** SGD (lr=0.001, momentum=0.9)
- ***Epoch Sayısı:*** 10

Eğitim süreci: Forward Propagation → Loss Hesaplama → Backpropagation → Ağırlık Güncelleme

4. Test ve Değerlendirme: Model eval() modunda test veri seti üzerinde değerlendirilmiş ve başarı metriği olarak Accuracy kullanılmıştır:

Accuracy = Doğru Tahmin Sayısı / Toplam Örnek Sayısı

CNN mimarisi, görüntü verilerindeki uzamsal özellikleri koruyarak öğrenme yaptığı için klasik tam bağlantılı ağlara kıyasla daha başarılı sonuçlar üretmektedir.

5. Sonuç:

Bu proje, PyTorch ile CNN mimarisinin temel prensiplerini uygulamalı olarak göstermektedir. Görüntü sınıflandırma problemlerinde evrişim katmanlarının etkinliğini ortaya koymakta ve daha gelişmiş derin öğrenme modelleri için temel oluşturmaktadır.

PROJE 3: Sinüs Dalgası Tahmini

PyTorch ile Yinelemeli Sinir Ağı (RNN) Uygulaması:

PyTorch kütüphanesi kullanılarak **Zaman Serisi Analizi** kapsamında bir Sinüs dalgasının gelecekteki değerlerini tahmin etmeyi amaçlamaktadır. Proje, ardışık verilerin işlenmesinde kullanılan Yinelemeli Sinir Ağları (RNN) mimarisinin temel çalışma prensiplerini ve uygulamasını göstermektedir.

1. Veri Hazırlama ve Kayan Pencere (Sliding Window) Yöntemi:

Zaman serisi verileri, modelin öğrenebileceği bir yapıya dönüştürülmüştür:

- **Veri Üretimi:** 0-100 aralığında 1000 adet sinüs dalgası noktası oluşturulmuştur.
- **Kayan Pencere:** Modelin bir sonraki adımı (t+1) tahmin edebilmesi için geçmişe dönük 50 adım (t – 49... t) girdi olarak kullanılmıştır.
- **Tensör Dönüşümü:** Veriler, PyTorch'un temel veri yapısı olan **Tensör** formatına ve [Batch, Sequence, Feature] boyutuna getirilmiştir.

2. Model Mimarisi: Many-to-One RNN

Model, ardışık verileri işleyip tek bir çıktı üreten "Çoktan-Bire" (Many-to-One) mimarisine sahiptir:

- **RNN Katmanı:** Girdi dizisindeki zamansal bağımlılıkları öğrenir.
- **Gizli Katman (Hidden Layer):** 16 nöronlu bir gizli durum (hidden state) yapısı kullanılmıştır.
- **Tam Bağlantılı (Linear) Katman:** RNN'in ürettiği son gizli durumu, tek bir sürekli değer olan tahmin çıktısına eşler.

3. Eğitim Süreci ve Hiperparametreler

Model, hatayı minimize etmek için gradyan tabanlı bir optimizasyon sürecinden geçmiştir:

- **Kayıp Fonksiyonu:** Regresyon problemleri için standart olan **MSELoss** (Ortalama Kare Hata) kullanılmıştır.
- **Optimizasyon:** Parametre güncellemeleri için **Adam** algoritması tercih edilmiştir.
- **Eğitim Döngüsü:** Veri seti 20 epoch boyunca taranmış; her adımda ileri besleme, kayıp hesaplama, **Autograd** ile backpropagation ve ağırlık güncelleme işlemleri yapılmıştır.

4. Test ve Değerlendirme

Eğitilen model, eğitim setinde hiç görmediği farklı zaman aralıklarındaki sinüs değerleri üzerinde test edilmiştir:

- **Inference (Çıkarım):** Model eval() moduna alınarak test verileri üzerinde tahminler üretilmiştir.
- **Görselleştirme:** Eğitim verisi, test verisi ve modelin ürettiği tahminler bir grafik üzerinde karşılaştırılarak modelin genelleme yeteneği doğrulanmıştır.

5. Projenin Önemi

Bu çalışma; PyTorch'un dinamik hesaplama grafiği yeteneğini zaman serisi gibi değişken doğalı verilerde nasıl kullandığını kanıtlar. Ayrıca, finansal tahminler veya anomali tespiti gibi daha karmaşık gerçek dünya senaryoları için bir temel teşkil eder.

PROJE 4: Kelime Bazlı Metin Üretimi

PyTorch ile LSTM (Long Short-Term Memory) Uygulaması:

PyTorch kütüphanesi kullanılarak Doğal Dil İşleme (NLP) kapsamında, verilen bir metinden öğrenme yaparak ardışık kelime tahmini (metin üretimi) yapmayı amaçlamaktadır. Proje, uzun vadeli bağımlılıkları hatırlama yeteneğine sahip olan LSTM (Long Short-Term Memory) mimarisinin metin verileri üzerindeki etkinliğini ve hiperparametre optimizasyon süreçlerini göstermektedir.

1. Veri Ön İşleme ve Hazırlık

Modelin eğitilebilmesi için ham metin verisi üzerinde şu işlemler uygulanmıştır:

- **Temizleme ve Tokenizasyon:** Noktalama işaretleri kaldırılmış, tüm metin küçük harfe çevrilmiş ve kelime birimlerine (token) ayrılmıştır.
- **Sayısal Temsil (Indexing):** Her benzersiz kelimeye bir indeks atanarak "Kelime → İndeks" ve "İndeks → Kelime" sözlükleri (vocab) oluşturulmuştur.
- **Girdi-Hedef Çiftleri:** Metindeki her kelime bir "Girdi", ondan sonra gelen kelime ise "Hedef" olarak eşleştirilmiştir.
- **Tensör Dönüşümü:** Kelime indeksleri, PyTorch'un temel veri yapısı olan Tensör formatına dönüştürülmüştür.

2. Model Mimarisi: LSTM Yapısı

Model, hiyerarşik bir API yapısı kullanılarak üç temel katmandan inşa edilmiştir:

- **Embedding (Gömme) Katmanı:** Kelime indekslerini, anlamsal ilişkileri temsil eden yoğun vektör (dense vector) formatına dönüştürür.
- **LSTM Katmanı:** Metindeki kelime sırasını ve bağlamı öğrenerek gizli durum (hidden state) bilgisini günceller.
- **Linear (Tam Bağlantılı) Katman:** LSTM'den gelen veriyi kelime haznesi boyutuna eşleyerek bir sonraki kelimenin ne olacağını tahmin eder.

3. Hiperparametre Optimizasyonu (Grid Search)

Modelin en yüksek performansa ulaşması için **Grid Search** yöntemi uygulanmıştır:

- **Denenen Parametreler:** Farklı gömme boyutları (8, 16), gizli katman nöron sayıları (32, 64) ve öğrenme oranları (0.01, 0.005) sistematik olarak test edilmiştir.
- **Seçim Kriteri:** En düşük **MSELoss** veya **CrossEntropyLoss** değerini veren parametre kombinasyonu "En İyi Model" (Best Model) olarak belirlenmiştir.

4. Eğitim ve Çıkarım (Inference)

- **Eğitim: Adam** optimizasyon algoritması ve **Autograd** mekanizması kullanılarak geriye yayılım (backpropagation) ile ağırlıklar güncellenmiştir.
- **Tahmin:** Eğitilen model eval() moduna alınarak, bir başlangıç kelimesinden yola çıkıp ardışık kelime tahminleri yaparak yeni cümleler oluşturmaktadır.

5. Projenin Önemi

Bu çalışma, PyTorch'un dinamik hesaplama grafiği yeteneğinin kelime bazlı dizisel verilerde nasıl kullanıldığını kanıtlar. Büyük dil modellerinin (LLM) temel taşı olan LSTM ve Embedding mantığını uygulamalı olarak göstererek, daha karmaşık NLP projeleri için bir temel teşkil eder.

PROJE 5: MNIST El Yazısı Rakam Üretimi

PyTorch ile Çekişmeli Üretici Ağlar (GAN) Uygulaması:

PyTorch kütüphanesi kullanılarak derin öğrenmenin önemli alanlarından biri olan Çekişmeli Üretici Ağlar (GAN) mimarisini uygulamalı olarak göstermektedir. Projenin temel amacı, MNIST veri setindeki rakamları kopyalamak değil, bu veri setinden öğrendiği stil ve yapıları kullanarak tamamen yeni ve daha önce var olmamış el yazısı rakam görüntüleri üretmektir.

1. Veri Hazırlama ve Ön İşleme

Projenin ilk aşamasında MNIST veri seti torchvision.datasets üzerinden yüklenmiştir.

- **Normalizasyon:** Görüntü pikselleri transforms.Normalize((0.5,), (0.5,)) işlemi ile -1 ile 1 arasına çekilmiştir. Bu, Generator modelinin çıkışındaki Tanh aktivasyon fonksiyonu ile uyum sağlamak için kritiktir.
- **Batch Yapısı:** Veriler, eğitim stabilitesini artırmak için 128'lik paketler (batch) halinde DataLoader yapısına dönüştürülmüştür.

2. Model Mimarisi: Çekişmeli İki Ağ

- **Generator (Üretici):** 100 boyutlu rastgele bir gürültü (latent space) vektörünü girdi olarak alır. ReLU aktivasyonlu gizli katmanlardan geçerek 784 pikselden (28x28) oluşan gerçekçi bir görüntü üretmeye çalışır.
- **Discriminator (Ayırt Edici):** Bir sınıflandırıcı görevi görür. Girdiği görüntünün "Gerçek" (MNIST veri setinden) mi yoksa "Sahte" (Generator tarafından üretilmiş) mi olduğunu ayırt eder. Çıkışta Sigmoid kullanarak 0 ile 1 arasında bir olasılık değeri üretir.

3. Eğitim Süreci ve Oyun Teorisi

Eğitim süreci, bu iki ağın birbirini geliştirdiği bir "kedi-fare" oyununa benzer:

- **Kayıp Fonksiyonu:** İki sınıflı bir problem olduğu için BCELoss (Binary Cross Entropy) kullanılmıştır.
- **Optimizasyon:** Her iki model de Adam optimizasyon algoritması ile (öğrenme oranı 0.0002) güncellenir.
- **Dinamik Süreç:** 1. **Discriminator Eğitimi:** Gerçek görüntülere 1, sahte görüntülere 0 etiketi vererek ayırt etme yeteneğini geliştirir. 2. **Generator Eğitimi:** Kendi ürettiği sahte görüntülerin Discriminator tarafından "Gerçek" (1) olarak sınıflandırılmasını sağlamaya çalışır.

4. Donanım ve Performans

- **GPU Desteği:** Kod, torch.cuda.is_available() kontrolü ile NVIDIA GPU varsa hesaplamaları otomatik olarak hızlandırır.
- **Autograd Mekanizması:** PyTorch'un otomatik türev sistemi sayesinde karmaşık çekişmeli gradyan hesaplamaları sorunsuz bir şekilde yönetilir.

5. Projenin Önemi ve Çıktılar

Bu çalışma, PyTorch'un Dinamik Hesaplama Grafiği yeteneğinin en üst düzey uygulamalarından biridir. Eğitim sonunda model, rastgele gürültüden anlamlı rakamlar (0-9) üretebilir hale gelir. Bu mimari, günümüzdeki modern yapay zekâ (GenAI) ve Deepfake teknolojilerinin temel taşı olan üretken modellerin çalışma prensibini anlamak için kritik bir referanstır.

PROJE 6: Iris Çiçeği Sınıflandırma

PyTorch ile Radyal Tabanlı Fonksiyon Ağı (RBFN) Uygulaması:

PyTorch kütüphanesi kullanılarak klasik Iris veri seti üzerinde çiçek türlerinin sınıflandırılmasını amaçlamaktadır. Proje, standart tam bağlantılı katmanlar yerine, giriş verilerini doğrusal olmayan bir özellik uzayına taşımak için özel bir **RBF (Radial Basis Function)** katmanının PyTorch ile nasıl inşa edileceğini ve eğitileceğini göstermektedir.

1. Veri Hazırlama Süreci

- **Sayısal Dönüştürme:** Tür isimleri modelin işleyebileceği 0, 1, 2 etiketlerine dönüştürülür.
- **Standardizasyon:** Özellikler, mesafe temelli hesaplamalarda (RBF çekirdeği) kararlılığı sağlamak için normalize edilmiştir.
- **Tensör Dönüşümü:** Veriler, PyTorch'un temel yapısı olan Tensör formatına ve uygun veri tiplerine (float32 ve long) çevrilmiştir.
- **DataLoader Yapısı:** Veri seti %30 oranında test setine ayrılarak modelin genelleme yeteneği korunmuştur.

2. Model Mimarisi: RBFN Yapısı

Model, nn.Module sınıfından türetilmiş ve özel bir ileri besleme mekanizmasıyla tasarlanmıştır:

- **RBF Çekirdek Katmanı:** Giriş verisi ile model tarafından öğrenilen "merkezler" arasındaki benzerliği Gaussian Kernel kullanarak hesaplar.
- **Öğrenilebilir Parametreler:** Merkez noktaları (centers) ve çan eğrisi genişliği (beta), nn.Parameter olarak tanımlanarak eğitim sırasında optimize edilir.
- **Çıkış Katmanı:** RBF katmanından gelen 10 adet özellik, 3 sınıf çıktısına (Iris türleri) eşlenen bir doğrusal (Linear) katmandan geçer.

3. Eğitim Süreci

Modelin eğitimi, PyTorch'un dinamik hesaplama grafiği ve otomatik türev sistemi kullanılarak gerçekleştirilmiştir:

- **Kayıp Fonksiyonu:** Çok sınıflı sınıflandırma için CrossEntropyLoss tercih edilmiştir.
- **Optimizasyon Algoritması:** Parametre güncellemeleri için **Adam** algoritması kullanılmıştır.
- **Eğitim Döngüsü:** 100 epoch boyunca ileri yayılım, kayıp hesaplama, geriye yayılım (backpropagation) ve ağırlık güncelleme adımları tekrarlanmıştır.

4. Test ve Değerlendirme

Model eval() moduna alınarak test veri seti üzerinde performans analizi yapılmıştır:

- **Başarı Ölçütü:** Modelin başarısı Accuracy (Doğruluk Oranı) metriği ile ölçülmüştür.
- **Hesaplama Yöntemi:** Tahmin edilen sınıfların gerçek etiketlerle eşleşme oranı üzerinden modelin başarısı doğrulanmıştır.

5. Projenin Önemi ve Donanım Desteği

- **Esneklik:** Bu çalışma, PyTorch'un standart dışı katman ve fonksiyonların (RBF çekirdeği gibi) kolayca entegre edilmesine olanak tanıyan modüler yapısını kanıtlar.
- **Performans:** Kod, CUDA destekli bir GPU varlığında hesaplamaları otomatik olarak hızlandıracak şekilde yapılandırılmıştır.
- **Akademik Değer:** RBF ağlarının mesafe temelli öğrenme mantığını göstererek, daha karmaşık çekirdek yöntemleri ve derin öğrenme mimarileri için temel oluşturur.

PROJE 7: FashionMNIST Görüntü Yeniden İnşası

PyTorch ile Autoencoder (Oto-Kodlayıcı) Uygulaması:

PyTorch kütüphanesi kullanılarak FashionMNIST veri seti üzerindeki giysi görsellerinin düşük boyutlu bir "latent space" içinde sıkıştırılmasını ve ardından bu sıkıştırılmış veriden orijinal görüntünün minimum kayıpla geri üretilmesini amaçlamaktadır. Çalışma; veri sıkıştırma, gürültü giderme ve özellik çıkarımı süreçlerinin temel taşı Autoencoder mimarisini uygulamalı olarak sunar.

1. Veri Hazırlama Süreci:

Veri seti torchvision.datasets aracılığıyla indirilmiş ve şu ön işlemler uygulanmıştır:

- **Tensör Dönüşümü:** Görüntüler transforms.ToTensor() ile tensör formatına çevrilmiş ve piksel değerleri otomatik olarak [0, 1] arasına normalize edilmiştir.
- **Batch Yapılandırması:** Eğitim ve test verileri, DataLoader yapısı kullanılarak 128'lik paketler (batch) halinde düzenlenmiştir.
- **Hiperparametre Tanımlama:** Modelin öğrenme hızı (1e-3) ve epoch sayısı (50) gibi kritik parametreler belirlenmiştir.

2. Model Mimarisi: Autoencoder Yapısı

- **Encoder (Kodlayıcı):** 784 piksellik (28x28) giriş verisini önce 256'ya, ardından 64 birime indirgeyerek veriyi en önemli özelliklerine sıkıştırır.
- **Decoder (Çözücü):** 64 birimlik özeti tekrar 256'ya sonrasında orijinal boyut olan 784 piksele genişletir. Çıkışta kullanılan Sigmoid aktivasyonu, piksel değerlerini [0, 1] aralığında tutar.
- **Boyut Koruma:** Unflatten katmanı ile vektör halindeki veriler tekrar 28x28 görüntü formuna sokulur.

3. Eğitim Süreci ve Denetleme: Model, giriş ve çıkış arasındaki farkı minimize etmek için gradyan tabanlı bir süreçten geçmiştir:

- **Kayıp Fonksiyonu:** Yeniden inşa başarısını ölçmek için regresyon temelli **MSELoss (Ortalama Kare Hata)** kullanılmıştır.
- **Optimizasyon:** Parametre güncellemeleri için **Adam** algoritması tercih edilmiştir.
- **Early Stopping (Erken Durdurma):** Kayıptaki iyileşme durduğunda eğitimi otomatik olarak sonlandırarak aşırı öğrenmeyi (overfitting) engelleyen bir mekanizma entegre edilmiştir.
- **Autograd:** PyTorch'un otomatik türev sistemi ile tüm ağırlık güncellemeleri yönetilmiştir.

4. Test ve Performans Değerlendirmesi

Model eval() moduna alınarak test veri seti üzerinde görsel ve sayısal olarak değerlendirilmiştir:

- **Yapısal Benzerlik (SSIM):** Modelin başarısı sadece piksel farkıyla değil, insan gözünün algısına yakın olan SSIM metriği ile ölçülmüştür.
- **Görselleştirme:** Orijinal görüntüler ile modelin ürettiği (decoded) görüntüler yan yana getirilerek modelin genelleme yeteneği doğrulanmıştır.

5. Projenin Önemi ve Donanım Kullanımı

- **Dinamik Hesaplama:** Bu çalışma, PyTorch'un dinamik hesaplama grafiği yeteneğini görüntü sıkıştırma ve yeniden inşa süreçlerinde nasıl kullandığını kanıtlar.
- **GPU Entegrasyonu:** Kod, CUDA destekli donanım mevcutsa hesaplamaları otomatik olarak hızlandıracak şekilde yapılandırılmıştır.
- **Genişletilebilirlik:** Bu mimari, günümüzdeki modern üretken modellerin (VAE, GAN vb.) ve görüntü işleme algoritmalarının temelini oluşturur.

PROJE 8: Oxford Flowers 102 Çiçek Sınıflandırma

PyTorch ile Transfer Learning (MobileNetV2) Uygulaması:

PyTorch kütüphanesi kullanılarak 102 farklı çiçek türünün yüksek doğrulukla sınıflandırılmasını amaçlamaktadır. Projede, ImageNet veri seti üzerinde önceden eğitilmiş (pre-trained) MobileNetV2 mimarisi kullanılarak Transfer Learning yöntemi uygulanmıştır. Bu yaklaşım, sınırlı veriyle derin özellik çıkarımı yapılmasını ve eğitim süresinin minimize edilmesini sağlar.

1. Veri Hazırlama ve Veri Artırma (Data Augmentation)

Oxford Flowers 102 veri seti torchvision.datasets üzerinden yüklenmiş ve modelin genelleme yeteneğini artırmak için şu dönüşümler uygulanmıştır:

- **Veri Artırma:** Eğitim verilerine rastgele yatay çevirme (RandomHorizontalFlip), döndürme (RandomRotation) ve renk varyasyonları (ColorJitter) uygulanarak modelin farklı koşullara karşı dayanıklılığı artırılmıştır.
- **Normalizasyon:** Görüntüler 224x224 boyutuna getirilmiş ve piksel değerleri standart aralığa normalize edilmiştir.
- **DataLoader:** Bellek yönetimi için veriler 8'lik paketler (batch) halinde yüklenmiştir.

2. Model Mimarisi: Transfer Learning Yapısı

- **Pre-trained Backbone:** ImageNet üzerinde eğitilmiş ağırlıklar kullanılarak görsel özellik çıkarımı dondurulmuştur.
- **Sınıflandırıcı Güncelleme:** Modelin son katmanı (classifier), hedef veri setindeki 102 çiçek türüne yanıt verecek şekilde yeni bir doğrusal katmanla (nn.Linear) değiştirilmiştir.
- **Cihaz Desteği:** Hesaplamalar, donanım hızlandırma için otomatik olarak **CUDA (GPU)** üzerine taşınmıştır.

3. Eğitim Süreci ve Optimizasyon

- **Kayıp Fonksiyonu:** Çok sınıflı sınıflandırma için CrossEntropyLoss kullanılmıştır.
- **Optimizer ve Scheduler: Adam** algoritması tercih edilmiş ve eğitim ilerledikçe öğrenme oranını otomatik olarak azaltan StepLR mekanizması eklenmiştir.
- **Eğitim Döngüsü:** 3 epoch boyunca ileri besleme, kayıp hesaplama ve geriye yayılım (backward) işlemleri tqdm ilerleme çubuğu eşliğinde takip edilmiştir.

4. Test ve Performans Değerlendirmesi

Model eval() moduna alınarak doğrulama veri seti üzerinde kapsamlı bir analize tabi tutulmuştur:

- **Hata Matrisi (Confusion Matrix):** Tahmin edilen sınıflar ile gerçek sınıflar arasındaki ilişki Seaborn heatmap kullanılarak görselleştirilmiştir.
- **Sınıflandırma Raporu:** Precision, Recall ve F1-score metrikleri üzerinden her bir çiçek türü için performans raporu oluşturulmuştur.

5. Projenin Önemi ve Uygulama Alanları

- **Verimlilik:** MobileNetV2 kullanımı, modelin mobil ve düşük güç tüketimli cihazlarda çalışabilmesine olanak tanır.
- **Transfer Öğrenme:** Bu proje, büyük veri setlerinde öğrenilen bilgilerin spesifik alanlara (botanik, tarım vb.) nasıl başarıyla aktarılabileceğini kanıtlar.
- **Esneklik:** PyTorch'un modüler yapısı sayesinde farklı pre-trained modellerin (ResNet, EfficientNet vb.) sisteme entegre edilmesi oldukça kolaydır.

PROJE 9: CIFAR-10 Nesne Tanıma

PyTorch ile ResNet (Residual Network) ve Transfer Learning:

PyTorch kütüphanesi kullanılarak **Residual Network** üzerinden CIFAR-10 veri seti sınıflandırmasını amaçlamaktadır. Çalışma, hem mimariyi temelden inşa eden "Custom ResNet" yaklaşımını hem de önceden eğitilmiş modelleri kullanan "Transfer Learning" stratejisini bir arada sunar.

1. Veri Hazırlama ve Ön İşleme

CIFAR-10 veri seti, 10 farklı sınıfa ait 32x32 boyutunda renkli (RGB) görüntülerden oluşur. Veri hazırlığı sürecinde şu adımlar izlenmiştir:

- **Tensör Dönüşümü:** Görüntüler transforms.ToTensor() ile PyTorch'un temel veri yapısı olan tensörlere dönüştürülmüştür.
- **Normalizasyon:** Görüntü pikselleri, modelin daha kararlı öğrenmesi için 0.5 ortalama ve standart sapma ile [-1, 1] aralığına normalize edilmiştir.
- **DataLoader Yapılandırması:** Veriler, eğitim sırasında belleği verimli kullanmak adına 64'lük paketler (batch) halinde yüklenmiştir.

2. Model Mimarisi: Residual (Artık) Bloklar

ResNet'in klasik CNN mimarilerinden en büyük farkı, **Skip Connection** kullanımıdır.

- **Residual Block:** Derin ağlarda karşılaşılan gradyan kaybolması (vanishing gradient) problemini, girişi doğrudan çıktıya ekleyerek (F(x) + x) çözer.
- **Custom ResNet Yapısı:** Model; başlangıç evrişimi, ardından gelen 4 ana katman ve son olarak Global Average Pooling ile tam bağlantılı (Linear) bir çıkış katmanından oluşur.
- **Boyut Eşitleme (Downsample):** Giriş ve çıkış boyutları uyuşmadığında, 1x1 evrişimler kullanılarak boyutlar otomatik olarak eşitlenir.

3. Transfer Learning Stratejisi

Proje, sıfırdan eğitim alternatifi olarak PyTorch'un torchvision.models kütüphanesindeki hazır **ResNet18** modelini sunar:

- **Pre-trained Backbone:** ImageNet üzerinde eğitilmiş devasa bilgi birikimi (ağırlıklar) kullanılarak görsel özellik çıkarımı yapılır.
- **Fine-tuning:** Modelin son katmanı (classifier), 1000 sınıflık yapıdan CIFAR-10'un 10 sınıflık yapısına uygun olarak yeniden tasarlanmıştır.

4. Eğitim ve Performans Yönetimi

- **Kayıp Fonksiyonu:** Çok sınıflı sınıflandırma için standart olan CrossEntropyLoss kullanılmıştır.
- **Optimizasyon:** Ağırlık güncellemeleri için hız ve performans dengesi sağlayan **Adam** algoritması tercih edilmiştir.
- **İlerleme Takibi:** tqdm kütüphanesi ile eğitim döngüsündeki her epoch ve batch gerçek zamanlı olarak izlenir.

5. Projenin Önemi

- **Esneklik ve Modülerlik:** PyTorch'un nn.Module yapısının, karmaşık ve özel katmanların (Residual Block gibi) inşasındaki kolaylığını kanıtlar.
- **Akademik ve Endüstriyel Temel:** Otonom sürüş (Tesla) ve nesne tespiti (Uber) gibi gerçek dünya uygulamalarında kullanılan derin mimarilerin temel çalışma mantığını gösterir.

PROJE 10: Gymnasium CartPole-v1

PyTorch ile Derin Q-Öğrenme (DQN) Uygulaması:

Bu çalışma, Pekiştirmeli Öğrenme alanının temel taşlarından biri olan Deep Q-Network (DQN) algoritmasının uygulanmasını amaçlar. Proje kapsamında, bir platform üzerinde dengede tutulması gereken bir direk (CartPole) problemi ele alınmış ve ajanın en iyi hareket stratejisini tamamen kendi deneyimlerinden öğrenmesi sağlanmıştır.

1. Ortam ve Deneyim Hafızası (Replay Memory): Ajanın geçmişteki hatalarından ders çıkarabilmesi ve istikrarlı bir öğrenme süreci geçirmesi için şu yapılar kurulmuştur:

- **Replay Memory:** Ajanın gerçekleştirdiği eylemleri (durum, aksiyon, ödül, sonraki durum) bir deque yapısında depolar.
- **Experience Replay:** Eğitim sırasında veriler ardışık değil, hafızadan rastgele örneklenerek seçilir. Bu, veriler arasındaki zamansal korelasyonu kırarak eğitimin kararlılığını artırır.

2. Model Mimarisi: Derin Q-Ağı: Model, durum bilgilerini girdi olarak alıp her bir olası aksiyon için beklenen toplam ödülü (Q-değeri) tahmin eden üç katmanlı bir yapay sinir ağıdır:

- **Giriş Katmanı:** Gözlem alanındaki (n_observations) fiziksel değerleri alır.
- **Gizli Katmanlar:** 128 nöronlu iki adet tam bağlantılı (Linear) katmandan oluşur ve aktivasyon fonksiyonu olarak **ReLU** kullanır.
- **Çıkış Katmanı:** Ajanın yapabileceği hamle sayısı (n_actions) kadar çıktı üretir.

3. Eğitim Stratejisi ve Hiperparametreler: Modelin öğrenme süreci, keşfetme (Exploration) ve mevcut bilgiyi kullanma (Exploitation) dengesi üzerine kuruludur:

- **Epsilon-Greedy Stratejisi:** Ajan başlangıçta rastgele hareketlerle ortamı keşfederken (eps_start), zamanla öğrendiği en iyi hamlelere odaklanır (eps_end).
- **Target Network (Hedef Ağ):** Eğitim stabilitesini sağlamak için iki ayrı ağ kullanılır (policy_net ve target_net). Hedef ağ, ana ağın ağırlıklarıyla **Soft Update (Tau)** yöntemiyle yavaşça güncellenir.
- **Kayıp Fonksiyonu:** Aykırı değerlere karşı daha dayanıklı olan **SmoothL1Loss** (Huber Loss) tercih edilmiştir.

4. Optimizasyon ve Donanım Kullanımı

- **Gradyan Kırpma (Clipping):** clip_grad_value_ fonksiyonu ile gradyanların aşırı büyümesi (exploding gradients) engellenerek eğitim güvenli sınırlarda tutulur.
- **GPU Entegrasyonu:** Hesaplamalar, torch.cuda kontrolü ile NVIDIA GPU üzerinde koşturularak tensör işlemleri hızlandırılmıştır.
- **Otomatik Türev:** PyTorch'un autograd mekanizması, karmaşık Bellman denklemi hesaplamalarında gradyanları otomatik olarak yönetir.

5. Projenin Önemi ve Çıktılar

- **Dinamik Karar Verme:** Bu çalışma, PyTorch'un dinamik hesaplama grafiği yeteneğinin anlık geri bildirim alan sistemlerdeki gücünü gösterir.
- **Görselleştirme:** Eğitim süreci boyunca bölüm süreleri (Duration) grafiklenerek, ajanın direği ne kadar süre dengede tutabildiği gerçek zamanlı olarak izlenebilir.
- **Temel Referans:** Bu mimari; robotik, otonom kontrol sistemleri ve gelişmiş yapay zekâ oyun stratejilerinin geliştirilmesinde kritik bir referans noktasıdır

PROJE 11: Duygu Analizi (Sentiment Analysis)

PyTorch ile Transformer Mimarisi Uygulaması:

PyTorch kütüphanesi ile metin tabanlı veriler üzerinde pozitif ve negatif duygu sınıflandırması yapmayı amaçlar. Projenin merkezinde, geleneksel ardışık modellerin aksine tüm kelimeleri aynı anda işleyebilen ve aralarındaki bağlamsal ilişkileri kurabilen Transformer mimarisi yer almaktadır.

1. Veri Ön İşleme ve Sayısal Temsil

Metin verilerinin model tarafından işlenebilmesi için bir dizi hazırlık aşamasından geçirilmiştir:

- **Temizleme (Preprocessing):** Cümleler küçük harfe çevrilmiş ve noktalama işaretlerinden arındırılarak standartlaştırılmıştır.
- **Kelime Dağarcığı (Vocabulary):** Eğitim setindeki benzersiz kelimelerden bir sözlük oluşturulmuş ve her kelimeye sayısal bir indeks atanmıştır.
- **Dolgu (Padding):** Farklı uzunluktaki cümlelerin aynı boyutta tensörlere dönüştürülmesi için <PAD> token'ı kullanılarak veriler sabit bir uzunluğa (max_len=15) getirilmiştir.

2. Model Mimarisi: Transformer Yapısı

Model, metindeki uzun vadeli bağımlılıkları yakalamak için hiyerarşik bir yapı kullanır:

- **Embedding ve Positional Encoding:** Kelime indeksleri vektör formatına (embedding_dim=32) dönüştürülür. Transformer mimarisi kelime sırasını doğal olarak bilmediği için, her kelimenin cümledeki konumunu belirten bir "konumsal kodlama" (Positional Encoding) eklenir.
- **Self-Attention (Öz-Dikkat):** nn.Transformer modülü, cümledeki her kelimenin diğer tüm kelimelerle olan ilişkisini hesaplayarak bağlamı öğrenir.
- **Sınıflandırma Kafası (Classification Head):** Transformer çıktısı düzleştirilerek tam bağlantılı (Linear) katmanlara aktarılır.

3. Eğitim Süreci ve Hiperparametreler

- **Kayıp Fonksiyonu:** İki sınıflı problemlerde uygun olan Binary Cross Entropy Loss kullanılır.
- **Optimizasyon:** Model parametrelerini güncellemek için **Adam** algoritması (lr=0.0005) tercih edilmiştir.
- **Aktivasyon:** Çıkış katmanında kullanılan **Sigmoid** fonksiyonu, modelin tahminini 0 ile 1 arasında bir olasılık değerine dönüştürür.

4. Başarı Ölçümü ve Değerlendirme

- **Accuracy (Doğruluk):** Modelin başarısı, doğru tahmin edilen duygu etiketlerinin toplam örnek sayısına oranı ile hesaplanmıştır.
- **Eşik Değeri (Threshold):** Sigmoid çıktısı 0.5'ten büyük olan tahminler pozitif (1), küçük olanlar negatif (0) olarak sınıflandırılmıştır.

5. Projenin Önemi ve Uygulama Alanları

- **Bağlamsal Öğrenme:** Transformer yapısı sayesinde model, belirgin kelimelerin yanı sıra kelimelerin cümle içindeki dizilişinden gelen anlam farklarını da yakalayabilir.
- **Genişletilebilirlik:** Bu çalışma, günümüzün modern Büyük Dil Modelleri (LLM) ve çeviri sistemlerinin temel taşı olan Transformer mekanizmasını uygulamalı olarak kanıtlar.
- **Modülerlik:** PyTorch'un esnek API'sı, farklı başlık sayıları (num_heads) ve katman derinlikleri (num_layers) ile modelin kolayca optimize edilmesine olanak tanır.